

Projet SHADOW

Pilotage d'une trottinette Ecoxtrém M41 (ESC CHK2-K1-03) par STM32 Nucleo F401RE en vue d'un robot autonome (skid-steer, 2 trottinettes)
Cahier des charges technique — révision du 2026-06-08 — contact@iris-systems.tn

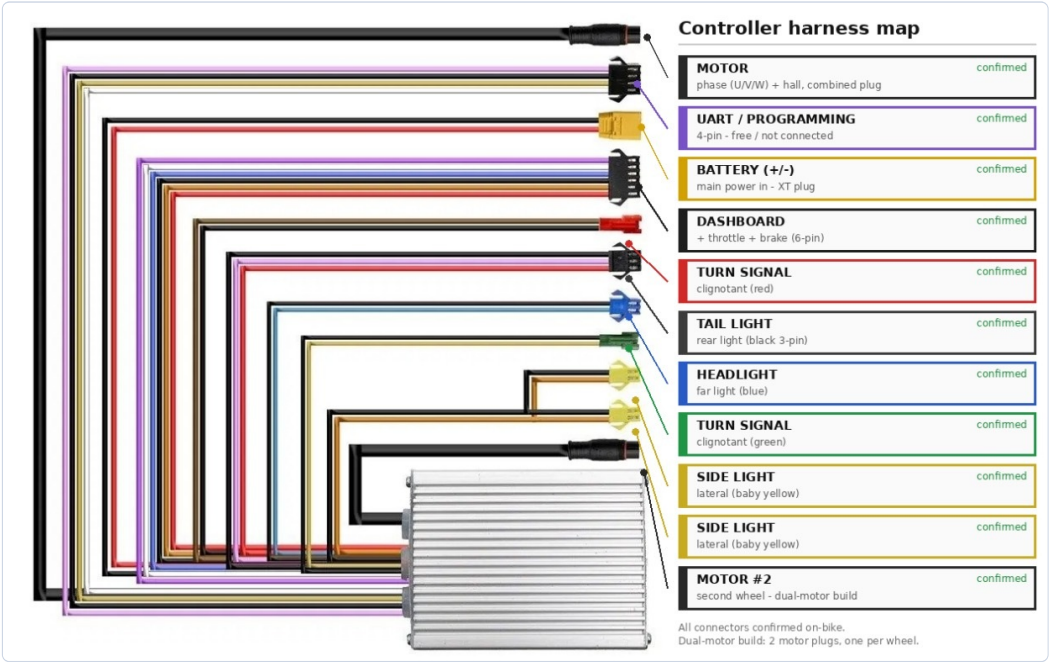


Figure 1. Faisceau de la trottinette — repérage des fils (voir le détail des rôles au §2).

1. Objectif du projet

Construire un **robot autonome à entraînement différentiel (skid-steer)** à partir de **2 trottinettes Ecoxtrém M41**. Chaque trottinette doit être pilotée **intégralement par une carte STM32**, sans intervention humaine sur le guidon, en reproduisant **toutes les fonctions** :

Fonction	Support physique	État
Accélération / vitesse moteur	Fil gris/blanc (throttle analogique)	✓ FAIT
Boucle fermée de vitesse (régulation)	Throttle + télémétrie bleu	✓ FAIT
Frein	Fil violet (bus commandes)	✓ FAIT
Feu / phare	Fil violet	✓ FAIT
Mode S / D (Sport / Drive)	Fil violet	✓ FAIT
Gear 1 / 2 / 3	Fil violet	✓ FAIT
Clignotants gauche / droit	Fil violet	✓ FAIT
Lecture vitesse (feedback)	Fil bleu (télémétrie)	✓ FAIT
Téléopération clavier	PC → STM32 (USB série)	✓ FAIT (tools/teleop.py)
Marche arrière	Relais d'inversion (à venir)	✗ À FAIRE
2e trottinette + différentiel	2e STM32 / coordination	✗ À FAIRE

BILAN : toutes les fonctions d'une trottinette unique sont aujourd'hui **pilotables depuis la carte**. Le « mur du violet » qui bloquait frein/feux/clignotants depuis des semaines a été **résolu** le 2026-06-06. Restent l'inversion (marche arrière) et la mise en robot 2 roues motrices.

2. Architecture électrique de la trottinette

La trottinette n'a **pas** un seul bus : elle a 3 canaux de signal distincts entre l'**afficheur** (guidon) et le **contrôleur** (ESC), plus l'alimentation.

Fil	Rôle	Nature du signal	Sens
Gris / blanc	Throttle (accélérateur)	Analogique 0,8 V (repos) → 3,2 V (plein gaz)	guidon → contrôleur
Violet	Bus de commandes (frein, feu, mode, gear, clignotants)	Série numérique 9600 8N1 , trame 20 octets @10/s, niveau 5 V open-drain	afficheur → contrôleur (unidirectionnel)
Bleu	Bus de télémétrie (vitesse, état)	Série numérique 9600 , trame 14 octets @10/s, header 0x02	contrôleur → afficheur
Noir	Masse (GND)	0 V de référence	commun
Rouge	Batterie +52 V	PUISSANCE	—
Orangé	+52 V commuté = ENABLE contrôleur	PUISSANCE	—

⚠ **SÉCURITÉ ABSOLUE** — Ne **JAMAIS** connecter le STM32 au rouge ni à l'orangé (52 V). Cela détruirait la carte. Le STM32 ne touche que : gris/blanc, violet, bleu, noir. **Roue toujours surélevée** pendant les tests moteur.

2.1 Le rôle réel de l'afficheur

L'afficheur du guidon a **deux rôles** qu'il a fallu séparer :

- **Interrupteur d'alimentation** (bouton power dessus, sur une ligne séparée) — indispensable pour **allumer** et **armer** le système.
- **Émetteur des commandes** sur le violet — c'est lui qu'on **remplace / double** par le STM32.

3. La carte STM32 — câblage et brochage

Carte : **STM32 Nucleo F401RE**. La navigation se fait par les **numéros Dx du header Arduino** (les étiquettes imprimées sur la carte).

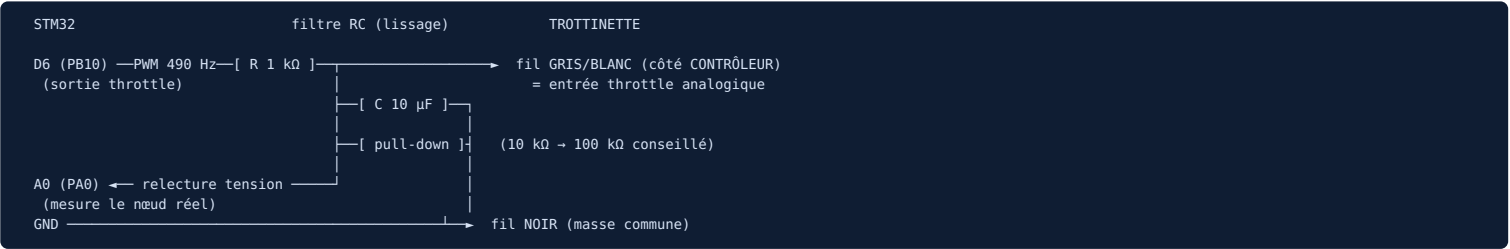
Étiquette Arduino	Pin STM32 réel	Connecté à	Fonction
D6	PB10	→ R 1 kΩ → nœud gris/blanc	Throttle (PWM 3,3 V) + condo 10 μF, pull-down
A0	PA0	← nœud gris/blanc	Relecture tension réelle du throttle (ADC 12 bits)
D8	PA9	↔ fil bleu (tap) / violet côté afficheur selon montage	Lecture télémétrie (vitesse) — UART half-duplex matériel
D9	PC7	← fil bleu (lecture parallèle)	Lecture télémétrie alternative (RX seul)
D10	PB6	→ fil violet côté contrôleur	Injection des commandes (USART1 half-duplex open-drain)
GND	—	↔ fil noir	Masse commune (obligatoire)
PC6 = D34	PC6	RIEN (morpho CN10)	⚠ PAS sur le header Dx — voir §5 (le bug !)

⚠ **Le piège de brochage qui a coûté des semaines** : sur le Nucleo F401RE, **PC6 n'est PAS l'étiquette D10**. D10 = **PB6**. PC6 n'existe que sur le connecteur morpho (D34). Tout le firmware qui « envoyait sur D10 » mais écrivait `PC6` parlait en réalité **dans le vide**. Voir la résolution complète §5.

3.1 Détail du câblage et rôle de chaque composant

Trois liaisons relient la carte à la trottinette, chacune avec son conditionnement. Voici le schéma complet puis le **pourquoi** de chaque pièce.

① Throttle (gris/blanc) — la seule voie avec des composants passifs



Composant	Valeur	Rôle	POURQUOI ce composant
R série	1 kΩ	1 ^{re} moitié du filtre RC	Le STM32 ne sort pas une tension continue : il sort un PWM (créneau 0/3,3 V à 490 Hz). Le contrôleur, lui, attend une tension analogique lisse comme celle d'une poignée. La résistance + le condo moyennent ce créneau en une tension continue ≈ rapport_cyclique × 3,3 V.
C (condensateur)	10 μF	2 ^e moitié du filtre RC (lissage)	Il emmagasine/restitue la charge entre deux fronts du PWM → écrase l'ondulation. Constante de temps τ = R·C = 1 kΩ × 10 μF = 10 ms → fréquence de coupure ≈ 1/(2πRC) ≈ 16 Hz , très en dessous des 490 Hz du PWM : le créneau est fortement aplani, le contrôleur voit une tension propre.
Pull-down	10 kΩ (→100 kΩ conseillé)	Référence basse + décharge	Donne un chemin de décharge au condo (la tension peut redescendre quand on lâche les gaz) et un état défini/fail-safe : si la carte se débranche, le nœud tombe vers 0. ⚠ Trop faible (10 k) il « tire » le nœud et l'atténue (nœud ≈ 0,8 × commande) → passer à 100 kΩ pour exploiter toute la plage 0,8–3,2 V.
A0 (relecture)	—	Mesure de la tension réelle	Comme le filtre + le pull-down atténuent un peu, on relit le nœud avec l'ADC pour connaître la vraie tension appliquée et la corriger (sert aussi à la boucle PI).

Pourquoi on COUPE le gris/blanc et on injecte côté contrôleur : si on laissait la poignée branchée, sa tension de repos (~0,8 V) « **tirerait** » la ligne et entrerait en conflit avec la sortie de la carte (deux sources sur le même fil). On coupe donc le fil et on injecte **uniquement côté contrôleur**. Évolution prévue (docs/06-08) : remplacer le PWM+RC par un **DAC MCP4725** + **buffer op-amp MCP6002** → tension 0–3,4 V parfaitement propre, sans ondulation ni atténuation.

② Violet (commandes) — pas de composant, mais un mode de sortie particulier



Élément	Réglage	POURQUOI
Open-drain	setHalfDuplex()	Le violet est de la logique 5 V (niveau repos ≈ 5,04 V imposé par le contrôleur). Un STM32 sort 3,3 V : en push-pull il ne pourrait jamais faire un « 1 » propre à 5 V, et il se battrait contre le pull-up du contrôleur. En open-drain , la carte ne fait que tirer à 0 (le « 0 ») et relâche pour le « 1 » → c'est le pull-up 5 V interne du contrôleur qui remonte la ligne à 5 V. Niveaux logiques corrects sans aucune résistance externe.
1 seul fil	half-duplex	Le bus violet est une liaison série sur un fil unique (TX et RX partagés). Le mode half-duplex single-wire du STM32 le gère nativement.

③ Bleu (télémétrie) — simple dérivation, aucun composant

Élément	Réglage	POURQUOI
Tap parallèle	on LIT, on ne coupe pas	Le bleu va du contrôleur vers l'afficheur. On se branche en dérivation (écoute) sans couper : ainsi l'afficheur continue de fonctionner (pas d'erreur E-06) et on lit la vitesse en parallèle.
TX factice hors ligne 5 V	RX sur PA11, jamais PC6	Si le TX factice de l'UART tombe sur un pin porté à 5 V (la ligne violet), le push-pull 3,3 V se bat contre et casse la réception (0 octet). Le TX factice doit rester sur un pin libre.

La masse commune (fil noir ↔ GND) est OBLIGATOIRE. Tous ces signaux sont des **tensions référencées à la masse**. Sans GND commun entre la carte et la trottinette, aucune lecture ni injection n'a de sens (références flottantes). ⚡ Rappel : jamais le rouge ni l'orangé (52 V).

3.2 Toolchain & flash

- Outil : `arduino-cli` + core `STMicroelectronics:stm32`. FQBN : `STMicroelectronics:stm32:Nucleo_64:pnum=NUCLE0_F401RE`.
- Flash = copier le fichier .bin sur le lecteur USB** /`media/manar/NUCLE0` (mass-storage ST-Link, pas besoin de CubeProgrammer). Script : `./flash_stm32.sh <nom>`.
- Console PC ↔ carte : port série `/dev/ttyACM0` @ 115200 (ouvrir avec `dtr=False, rts=False`).

Le throttle n'est PAS dans le violet (accélérer ne change aucun octet) → confirme qu'il reste analogique sur gris/blanc.

Série 9600, trame de 14 octets @10/s. **Header 0x02**. Trame roue arrêtée :

5. Problèmes rencontrés & solutions tes idées en bleu

Cette section retrace les blocages dans l'ordre, avec la **cause réelle** et la **solution**. Les **encadrés bleus** marquent **tes idées de manip** qui ont fait avancer ou débloquer le projet.

Problème A — Le throttle STM32 ne fait pas assez tourner la roue

Symptôme : commande 1,5 V → roue immobile.

Cause : le STM32 sort 3,3 V (vs 5 V de l'Arduino) ; avec la résistance + pull-down, le nœud n'atteint que ~0,8 × la commande. Il faut **nœud > ~1,3-1,4 V** pour démarrer.

Ton constat : « le voltage est le facteur clé du démarrage ». Confirmé : cmd 2,0 V → nœud 1,61 V → roue à 8,6 mph ✓.

Solution : commander plus haut (≥ 2,0 V). Amélioration possible : pull-down 10 k → 100 k pour réduire l'atténuation. **RÉSOLU**.

Problème B — Le repos throttle à 0 V provoque une faute

Cause : 0 V est interprété comme une **faute capteur Hall** de la poignée.

Solution : repos = **0,8 V jamais 0 V**, + **rampe douce** (0,5 V/s) au lieu d'un saut. **RÉSOLU**.

Problème C — « Le mur du violet » : couper le violet verrouille le contrôleur

Symptôme : dès qu'on coupait le violet pour s'insérer, l'afficheur passait en E07 et le contrôleur se mettait en sécurité (roue immobile). Relais, émulation, injection : **tout échouait**. Ce mur a duré des jours.

Fausse pistes éliminées (par mesure, pas au hasard)

- Octets / checksum exacts → corrects, mais immobile.
- Sens RX/TX, latence store-and-forward, relais cut-through push-pull → tous testés, tous immobiles.
- Bus bidirectionnel (afficheur attendrait une réponse) → **réfuté** : le contrôleur est **muet** sur le violet (0 trame côté contrôleur) → violet strictement unidirectionnel.
- Niveau 3,3 V vs 5 V → hypothèse forte un temps, gérée par open-drain + pull-up 5 V du contrôleur.

Ton idée décisive n°1 : « et si je coupe le violet SANS couper la batterie, pour conserver l'état de l'afficheur ? »
→ Test fait : violet coupé **à chaud** (contrôleur déjà armé) → **la roue tourne toujours** (throttle injecté, nœud 1,33 V). **Preuve capitale** : le contrôleur s'arme au boot avec le violet natif présent, puis **reste armé même violet coupé** tant qu'on ne power-cycle pas la batterie. C'est le **power-cycle à froid SANS violet natif** qui verrouille, pas la coupure en soi.

Ton idée décisive n°2 : « tu ne peux pas envoyer la même information que l'afficheur envoie ? Comme ça le contrôleur pense que ça vient de l'afficheur. »
→ C'est exactement le principe d'**émulation d'afficheur** retenu : le STM32 ne relaie plus, il **génère** les trames de commande comme si c'était l'afficheur. Concept juste — il ne manquait que le bon pin (voir Problème D).

Ton idée décisive n°3 : « contrôler tout ça avec un fil parallèle au violet relié au STM » + « ramène le projet original (dossier v2) qui faisait tourner les moteurs ».
→ Le projet Arduino original ne coupait **JAMAIS** le violet (doc : « violet... laisser branché »). Recette de récupération validée : **violet natif jamais coupé** (il arme le contrôleur) + **bleu tapé en parallèle** (lecture) + throttle injecté côté contrôleur. Mesuré : nœud 1,34 V → période bleu 60000→1116 (roue tourne) ; retour 0,8 V → 60000 (stop). **Scooter récupéré, afficheur SANS aucune erreur**.

Problème D — LA VRAIE CAUSE DU MUR : un mauvais pin (PC6 ≠ D10)

RÉSOLUTION 2026-06-06. Tous les firmwares d'injection violet écrivaient sur **PC6**. Or sur le Nucleo F401RE, **PC6 n'est pas sur le header Arduino** (c'est un pin morpheo, D34). Le fil violet-contrôleur est branché sur **D10 = PB6**. L'injection partait donc **sur un pin dans le vide** et n'atteignait jamais le contrôleur. « **Le mur du violet** », **c'était ça depuis le début** — émulation/relais/injection/armement échouaient tous pour CETTE raison, pas par rejet du contrôleur.

Ton apport décisif : « je ne me souviens plus exactement... mais je me rappelle D8 et D10 » → « violet est dans D10 côté contrôleur et D8 côté afficheur ».
→ C'est ce souvenir précis du brochage **par étiquette Dx** qui a permis d'identifier l'écart : le code disait « PC6 » mais le fil était sur D10. En corrigeant **PC6 → PB6 (=D10)**, tout a fonctionné.

Résultat sur le bon pin (PB6 = D10), firmware violet_d10 :

Commande injectée sur D10	Effet observé	Confirmé
Frein = ON	La roue S'ARRÊTE	oui ✓
Frein = OFF + throttle	La roue repart	oui ✓
Feu = ON	Le phare s'allume	oui ✓
Clignotant droit	Clignote	oui ✓
Clignotant gauche	Clignote	oui ✓

Conclusion : sur le bon pin, **le contrôleur OBÉIT au violet injecté**. Frein / feu / mode / gear / clignotants sont tous pilotables depuis la carte. **MUR RÉSOLU**.

Problème D-bis — ✓ Le power-cycle batterie à froid NE verrouille PLUS MAJ 2026-06-08

NOUVELLE PREUVE (toi, 2026-06-08) : « j'ai débranché la batterie puis je l'ai rebranchée, et tout marche sans erreur. » → **Le verrou au démarrage à froid a DISPARU**. On peut maintenant couper et rebrancher la batterie : la trottinette redémarre et obéit au STM **sans aucune erreur (pas de E07)**.

Ton interprétation : « au tout début, quand le contrôleur reçoit la commande du STM pour la première fois, il s'y habitue — ensuite il la connaît toujours. » → Observation confirmée : une fois que la carte a piloté le contrôleur une première fois, les power-cycles suivants repartent proprement.

Pourquoi ça marche maintenant (mécanisme) : ce qui verrouillait avant, ce n'était PAS la coupure batterie en soi — c'était qu'à froid le contrôleur ne voyait **aucun violet valide** (l'injection partait sur le mauvais pin PC6, dans le vide). Depuis que le STM injecte des trames violet **correctes sur D10/PB6 dès le démarrage**, le contrôleur **s'arme directement à partir de la carte** à chaque boot — il n'a plus besoin du violet natif de l'afficheur pour s'armer. Donc le « il s'habitue » se traduit techniquement par : **le STM fournit lui-même la trame d'armement valide à chaque allumage**.

Conséquence pour le robot : c'est une excellente nouvelle — plus besoin de séquence d'armement par l'afficheur, ni de bascule relais après boot. Il suffit que le STM **soit alimenté et émette déjà ses trames violet sur D10 au moment où la batterie est mise sous tension**. À garantir : la carte démarre vite et envoie le violet avant/pendant l'allumage du contrôleur.

Problème E — La lecture du bleu donnait 0 octet

Cause : un TX factice avait été mis sur **PC6**, qui porte la ligne violet tirée à 5 V ; le TX push-pull 3,3 V se battait contre → réception cassée. **Solution :** déplacer le TX factice hors de PC6 (`HardwareSerial blueUart(PC7, PA11)`). **RÉSOLU.**

Problème F — Flash qui ne « prend » pas

Le flash par mass-storage est parfois capricieux (ancien firmware encore en cours). **Solution :** reflasher et **vérifier la bannière série** avant tout test. **CONTOURNÉ.**

6. État du firmware & des outils

Fichier	Rôle	État
firmware_stm32/violet_d10	FIRMWARE PRINCIPAL : injection violet sur PB6/D10 (frein/feu/mode/gear/clignos) + throttle D6	✓ marche
firmware_stm32/throttle_clean	Throttle seul propre (repos 0,8 V, rampe) — récupération scooter	✓ marche
firmware_stm32/throttle_pi	Boucle fermée PI de vitesse (c<mph> tient la vitesse)	✓ marche
firmware_stm32/bus_watch / bus_sniff2	Lecture/tap des bus violet + bleu (décodage)	✓ marche
tools/teleop.py	Téléopération clavier (drive/frein/feu/mode/clignos/gear)	✓ prêt
tools/pi_keyboard_stm32.py	Clavier de test de la boucle PI	✓
violet_emul , violet_ctrl , violet_diag , relais...	Anciens firmwares d'injection sur PC6 (mauvais pin)	à corriger PC6→PB6 si réutilisés

6.1 Téléopération clavier — tools/teleop.py

À lancer dans un **vrai terminal** (le clavier doit être lu en direct) :

```
cd ~/Téléchargements/shadow-scooter-control-v2/shadow-scooter-control
python3 tools/teleop.py
```

Touche	Action	Touche	Action
a / z	accélérer / ralentir (±0,1 V)	f	feu (bascule)
x	repos (0,8 V)	s / d	mode S / mode D
ESPACE	frein (bascule)	g / h	cligno gauche / droit
1 / 2 / 3	gear	p	quitter (remet le repos)

Rappel : le paste dans un terminal Linux = **Ctrl+Shift+V** (pas Ctrl+V).

7. Chaîne de contrôle validée (résumé pour le robot)

Fonction robot	Voie	Pin STM32
Accélération	Throttle analogique (gris/blanc)	D6 (PB10) + relecture A0
Régulation de vitesse	Boucle PI (throttle ↔ bleu)	D6 + D8/D9
Frein / Feu / Mode / Gear / Clignos	Injection violet	D10 (PB6)
Vitesse (feedback)	Lecture télémétrie bleu	D8 (PA9) ou D9 (PC7)
Masse	commune	GND ↔ noir

Procédure d'armement (simplifiée — validée 2026-06-08) : le STM32 alimenté **injecte ses trames violet valides sur D10/PB6 dès le démarrage** → à la mise sous tension de la batterie, le contrôleur **s'arme directement depuis la carte**, sans erreur, **même après un power-cycle batterie à froid**. Plus besoin de la séquence d'armement par l'afficheur ni d'une bascule relais après boot. **Seule condition** : que la carte soit déjà alimentée et émette le violet quand le contrôleur s'allume. L'afficheur ne sert plus qu'au bouton power (alimentation).

Note historique : la version précédente de ce document exigeait un armement par l'afficheur puis une bascule relais — c'était nécessaire **uniquement** tant que l'injection partait sur le mauvais pin (PC6). Depuis la correction D10/PB6, cette contrainte est levée.

8. Ce qu'il reste à faire (tes prochaines étapes)

- Téléopération clavier en conditions réelles** — lancer `teleop.py` et valider tous les contrôles enchaînés (en cours).
- Marche arrière** — implémenter l'inversion moteur (relais d'inversion, cf. docs/05). Seule fonction motrice manquante.
- Corriger les anciens firmwares** PC6 → PB6 si on veut réutiliser `violet_emul` / relais.
- 2e trottinette** — dupliquer la chaîne, puis coordonner les 2 (différentiel skid-steer) : vitesses/sens indépendants gauche/droite.
- Fiabilisation** — ressouder le nœud gris/blanc (faux-contacts), passer le pull-down à 100 k, sécuriser la séquence d'allumage.
- Couche autonomie** — au-dessus de la téléop : capteurs + logique de navigation pilotant les mêmes commandes.

9. Règles de sécurité permanentes

- 🚫 **Jamais le STM32 sur rouge ou orangé (52 V).**
 - Roue surélevée pour tout test moteur.
 - Throttle repos = 0,8 V, jamais 0 V (faute Hall).
 - Ouvrir le port série avec `dtr=False, rts=False` .
 - Au démarrage à froid, le STM doit émettre des trames violet valides sur D10 dès l'allumage (sinon, sans violet valide ni afficheur, le contrôleur ne s'arme pas).
 - Vérifier la télémétrie bleu avant de conclure « pas de rotation ».